COMSATS UNIVERSITY

ISLAMABAD

ATTOCK CAMPUS

ASSIGNMENT 01

SUBJECT:
Data Structure

ASSIGNMENT:
01

SUBMITTED BY:
Inza Bibi

REG NO:
FA25-BCS-031

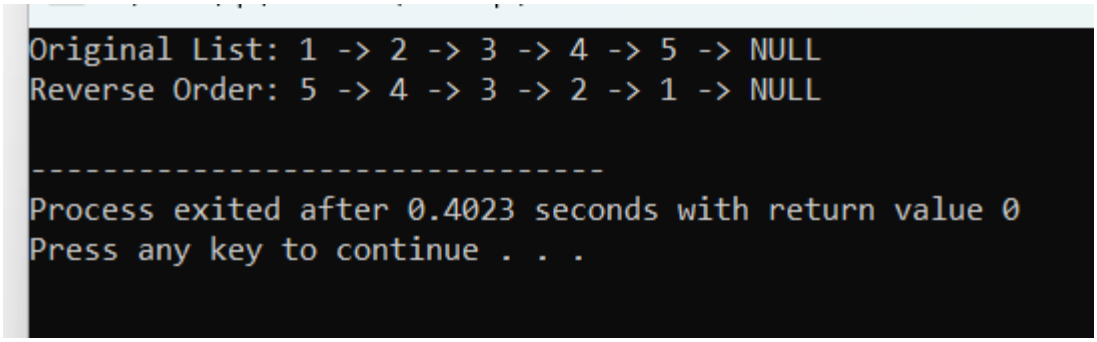
DATE:
October 4, 2026

Tasks of Singly Linked List:

Task 1: Display Linked List in Reverse Order

```
#include <iostream>
using namespace std;
// Node structure for Singly Linked List
struct Node {
   int data;      // Stores the value of the node
   Node* next;    // Points to the next node in the list
};
Node* head = NULL;  // Global head pointer, initially NULL
// Insert a new node at the end of the list
void insertEnd(int value) {
   Node* newNode = new Node;   // Create a new node in memory
   newNode->data = value;      // Store the value in the new node
   newNode->next = NULL;       // New node's next is NULL
   if (head == NULL) {         // If list is empty
      head = newNode;          // Make new node the head
      return;                  // Exit the function
   }
   Node* temp = head;          // Start from head
   while (temp->next != NULL) { // Traverse to the last node
      temp = temp->next;       // Move to next node
   }
   temp->next = newNode;       // Link last node to new node
}
```

```
// Display list in reverse order using recursion (does not modify structure)
void displayReverse(Node* temp) {
   if (temp == NULL) return;       // Base case: end of list
   displayReverse(temp->next);     // Recursive call: go to the end first
   cout << temp->data << " -> ";   // Print while returning back
}
// Main function
int main() {
   insertEnd(1);   // Insert 1 at end
   insertEnd(2);   // Insert 2 at end
   insertEnd(3);   // Insert 3 at end
   insertEnd(4);   // Insert 4 at end
   insertEnd(5);   // Insert 5 at end
   cout << "Original List: ";     // Print label
   Node* temp = head;             // Start from head
   while (temp != NULL) {         // Traverse the list
      cout << temp->data << " -> "; // Print current node's data
      temp = temp->next;         // Move to next node
   }
   cout << "NULL" << endl;        // End of list
   cout << "Reverse Order: ";     // Print label
   displayReverse(head);          // Call reverse display function
   cout << "NULL" << endl;        // End of list
   return 0;                      // Program ends successfully
}
```

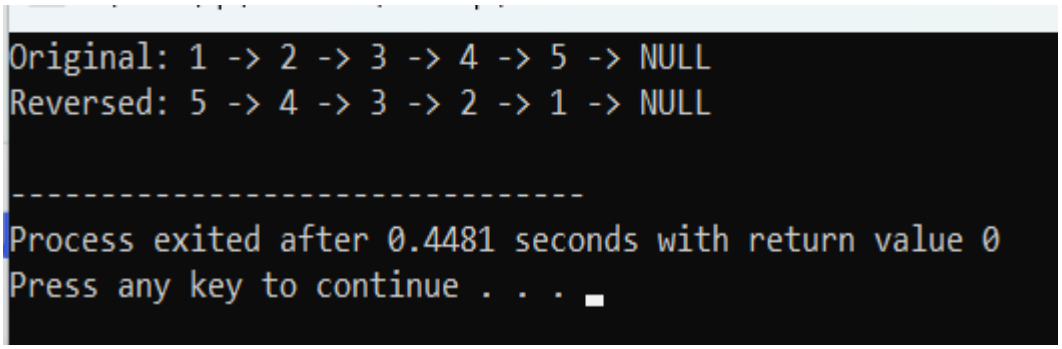


Task 2: Reverse the Linked List Itself

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;       // Stores the value
    Node* next;     // Points to the next node
};
Node* head = NULL;  // Global head pointer
// Insert at end
void insertEnd(int value) {
    Node* newNode = new Node;   // Create a new node
    newNode->data = value;      // Store the value
    newNode->next = NULL;       // Next is NULL
    if (head == NULL) {         // If list is empty
        head = newNode;         // New node becomes head
        return;
    }
    Node* temp = head;          // Start from head
    while (temp->next != NULL) temp = temp->next; // Go to last node
```

```
    temp->next = newNode;       // Link last node to new node
}
// Reverse the linked list (only links are changed)
void reverseList() {
    Node* prev = NULL;          // Previous node (starts as NULL)
    Node* current = head;       // Current node (starts from head)
    Node* next = NULL;          // Next node (temporary)
    while (current != NULL) {   // Traverse the entire list
        next = current->next;   // Store the next node
        current->next = prev;   // Reverse the link (point to previous)
        prev = current;         // Move prev forward
        current = next;         // Move current forward
    }
    head = prev;                // Update head to the new first node
}
// Display the list
void display() {
    Node* temp = head;          // Start from head
    while (temp != NULL) {      // Traverse the list
        cout << temp->data << " -> "; // Print current data
        temp = temp->next;      // Move to next node
    }
    cout << "NULL" << endl;     // End of list
}
// Main function
int main() {
```

```
    insertEnd(1);   // Insert 1
    insertEnd(2);   // Insert 2
    insertEnd(3);   // Insert 3
    insertEnd(4);   // Insert 4
    insertEnd(5);   // Insert 5
    cout << "Original: "; display();  // Display original list
    reverseList();                    // Reverse the list
    cout << "Reversed: "; display();  // Display reversed list
    return 0;
}
```



Task 3: Remove All Duplicate Contacts

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;      // Contact value
    Node* next;    // Pointer to next node
};
```

```
Node* head = NULL;  // Head pointer
// Insert at end
void insertEnd(int value) {
   Node* newNode = new Node;   // Create new node
   newNode->data = value;      // Store value
   newNode->next = NULL;       // Next is NULL
   if (head == NULL) {        // If list empty
      head = newNode;         // New node becomes head
      return;
   }
   Node* temp = head;          // Start from head
   while (temp->next != NULL) temp = temp->next; // Go to last node
   temp->next = newNode;       // Link to new node
}
// Remove all duplicates from the list
void removeDuplicates() {
   Node* current = head;       // Outer loop pointer
   while (current != NULL && current->next != NULL) {
      Node* runner = current; // Inner loop pointer
      while (runner->next != NULL) {
         if (runner->next->data == current->data) { // Duplicate found
            Node* duplicate = runner->next;       // Store duplicate
            runner->next = duplicate->next;       // Skip duplicate
            delete duplicate;                     // Free memory
         } else {
            runner = runner->next;                // Move forward
```

```
            }
        }
        current = current->next;    // Move to next node
    }
}
// Display the list
void display() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    } cout << "NULL" << endl; }
int main() {
    insertEnd(10); insertEnd(20); insertEnd(10);
    insertEnd(30); insertEnd(20); insertEnd(40); insertEnd(10);
    cout << "Original: "; display();    // Display original list
    removeDuplicates();                 // Remove duplicates
    cout << "After removing duplicates: "; display();
    return 0;
}
```

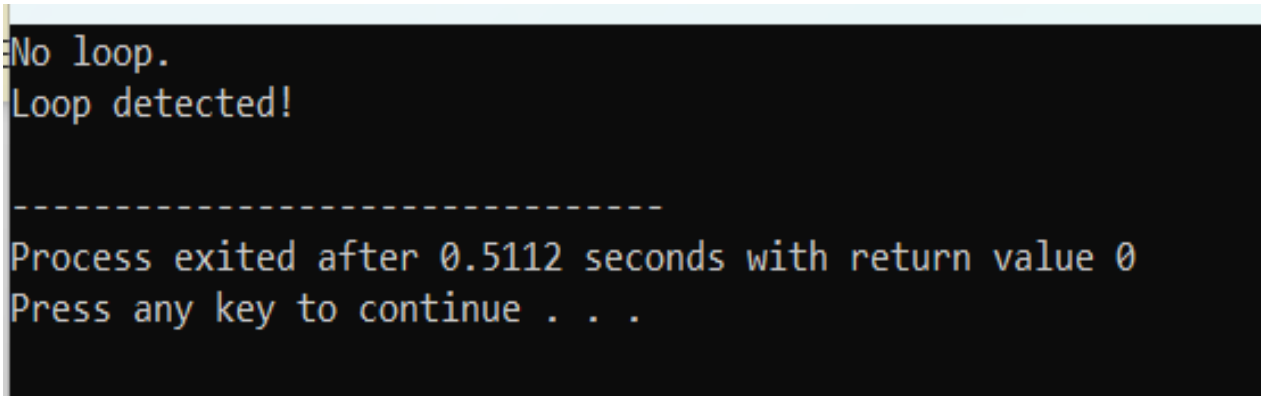
```
Original: 10 -> 20 -> 10 -> 30 -> 20 -> 40 -> 10 -> NULL
After removing duplicates: 10 -> 20 -> 30 -> 40 -> NULL

--------------------------------
Process exited after 0.3978 seconds with return value 0
Press any key to continue . . .
```

Task 4: Detect Loop in Linked List

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;
    Node* next;
};
Node* head = NULL;
// Detect loop using Floyd's Cycle Detection algorithm
bool detectLoop() {
    Node* slow = head;      // Slow pointer (1 step)
    Node* fast = head;      // Fast pointer (2 steps)
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;             // Move slow by 1 step
        fast = fast->next->next;       // Move fast by 2 steps
        if (slow == fast) return true;  // Loop detected
    }
    return false;                    // No loop
}
// Main function
int main() {
    // Create list: 1 -> 2 -> 3 -> 4 -> 5
    Node* n1 = new Node; n1->data = 1;
    Node* n2 = new Node; n2->data = 2;
    Node* n3 = new Node; n3->data = 3;
```

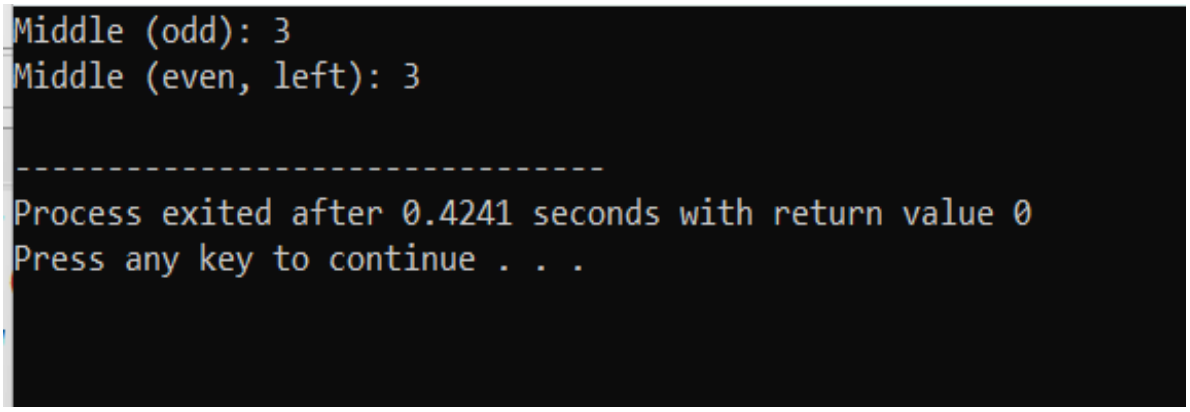
```
    Node* n4 = new Node; n4->data = 4;
    Node* n5 = new Node; n5->data = 5;
    head = n1;            // Set head
    n1->next = n2;        // Link nodes
    n2->next = n3;
    n3->next = n4;
    n4->next = n5;
    n5->next = NULL;      // Normal end
    if (detectLoop()) cout << "Loop detected!" << endl;
    else cout << "No loop." << endl;
    n5->next = n2;        // Create a loop: 5 -> 2
    if (detectLoop()) cout << "Loop detected!" << endl;
    else cout << "No loop." << endl;
    return 0;
}
```



Task 5: Find Middle Node

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
   int data;
   Node* next;
};
Node* head = NULL;
// Insert at end
void insertEnd(int value) {
   Node* newNode = new Node;
   newNode->data = value;
   newNode->next = NULL;
   if (head == NULL) { head = newNode; return; }
   Node* temp = head;
   while (temp->next != NULL) temp = temp->next;
   temp->next = newNode;
}
// Find middle node using slow and fast pointers
int findMiddle() {
   Node* slow = head;      // Slow pointer
   Node* fast = head;      // Fast pointer
   while (fast != NULL && fast->next != NULL) {
      slow = slow->next;         // 1 step
      fast = fast->next->next;    // 2 steps
```

```
    }
    return slow->data;            // Slow is at middle
}
// Main function
int main() {
    // Odd number of nodes
    insertEnd(1); insertEnd(2); insertEnd(3); insertEnd(4); insertEnd(5);
    cout << "Middle (odd): " << findMiddle() << endl;
    // Even number of nodes
    head = NULL;
    insertEnd(1); insertEnd(2); insertEnd(3); insertEnd(4);
    cout << "Middle (even, left): " << findMiddle() << endl;
    return 0; }
```

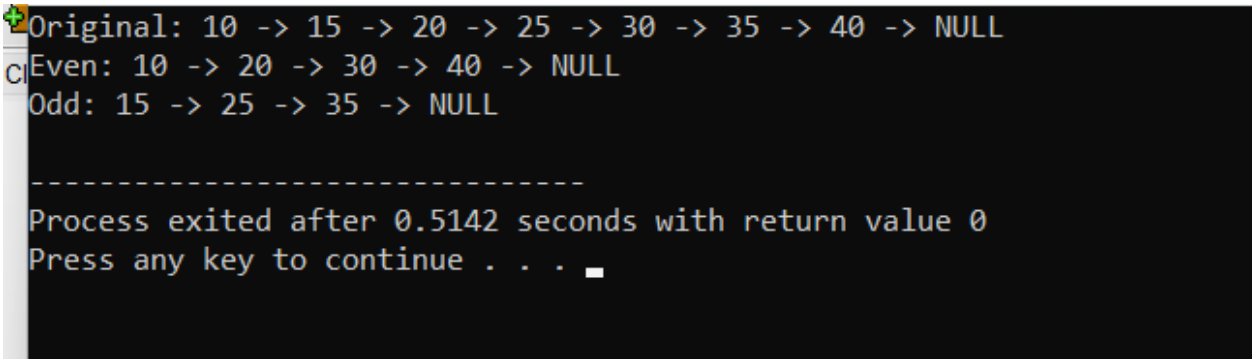


Task 6: Divide List into Even and Odd Values

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;
    Node* next;
};
Node* head = NULL;
// Insert at end
void insertEnd(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
}
// Divide list into even and odd lists
void divideEvenOdd(Node*& evenHead, Node*& oddHead) {
    evenHead = NULL;       // Even list head
    oddHead = NULL;        // Odd list head
    Node* evenTail = NULL;  // Even list tail
    Node* oddTail = NULL;   // Odd list tail
    Node* temp = head;      // Traverse original list
```

```
    while (temp != NULL) {
        if (temp->data % 2 == 0) {      // Even value
            if (evenHead == NULL) { evenHead = temp; evenTail = temp; }
            else { evenTail->next = temp; evenTail = temp; }
        } else {                        // Odd value
            if (oddHead == NULL) { oddHead = temp; oddTail = temp; }
            else { oddTail->next = temp; oddTail = temp; }
        }
        temp = temp->next;              // Move forward
    }
    if (evenTail != NULL) evenTail->next = NULL;  // End even list
    if (oddTail != NULL) oddTail->next = NULL;    // End odd list
}
// Display a list
void display(Node* h) {
    while (h != NULL) { cout << h->data << " -> "; h = h->next; }
    cout << "NULL" << endl;
}
// Main function
int main() {
    insertEnd(10); insertEnd(15); insertEnd(20); insertEnd(25);
    insertEnd(30); insertEnd(35); insertEnd(40);
    cout << "Original: "; display(head);
    Node* evenHead; Node* oddHead;
    divideEvenOdd(evenHead, oddHead);
    cout << "Even: "; display(evenHead);
```

```
    cout << "Odd: "; display(oddHead);
    return 0;
}
```



Task 7: Reverse First Half and Second Half Separately

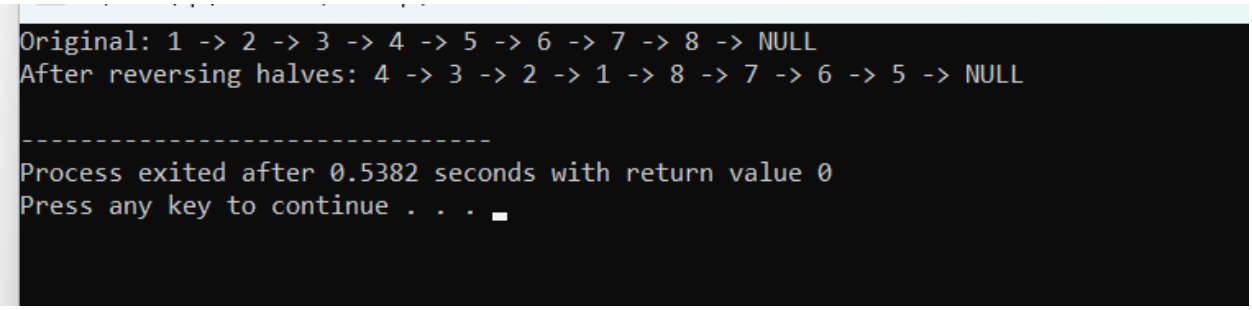
```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;
    Node* next;
};
Node* head = NULL;
// Insert at end
void insertEnd(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    Node* temp = head;
```

```
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
}
// Helper: Reverse a list starting from 'start'
Node* reverseList(Node* start) {
    Node* prev = NULL;
    Node* current = start;
    Node* next = NULL;
    while (current != NULL) {
        next = current->next;      // Store next
        current->next = prev;      // Reverse link
        prev = current;            // Move prev
        current = next;            // Move current
    } return prev;                 // New head
}
// Reverse both halves separately
void reverseHalves() {
    // Step 1: Find middle using slow and fast pointer
    Node* slow = head;
    Node* fast = head;
    Node* prevSlow = NULL;
    while (fast != NULL && fast->next != NULL) {
        prevSlow = slow;
        slow = slow->next;
      fast = fast->next->next;
    }
```

```
    Node* firstHalf = head;     // First half starts at head
    Node* secondHalf = slow;    // Second half starts at middle
    prevSlow->next = NULL;      // Break list into two halves
    // Step 2: Reverse both halves
    Node* newFirst = reverseList(firstHalf);
    Node* newSecond = reverseList(secondHalf);
    // Step 3: Connect both halves
    head = newFirst;
    Node* temp = newFirst;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newSecond;
}
// Display
void display() {
    Node* temp = head;
    while (temp != NULL) { cout << temp->data << " -> "; temp = temp->next; }
    cout << "NULL" << endl;
}

// Main function
int main() {
    insertEnd(1); insertEnd(2); insertEnd(3); insertEnd(4);
    insertEnd(5); insertEnd(6); insertEnd(7); insertEnd(8);
    cout << "Original: "; display();
    reverseHalves();
    cout << "After reversing halves: "; display();
```

return 0;}



Task 8: Remove Duplicate Stamps

```
#include <iostream>
#include <string>
using namespace std;
// Node structure
struct Node {
    string data;    // Stamp design
    Node* next;
};
Node* head = NULL;
// Insert at end
void insertEnd(string value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;}
```

```
// Remove duplicates
void removeDuplicates() {
   Node* current = head;
   while (current != NULL && current->next != NULL) {
      Node* runner = current;
      while (runner->next != NULL) {
         if (runner->next->data == current->data) {
            Node* duplicate = runner->next;
            runner->next = duplicate->next;
            delete duplicate;
         } else {
            runner = runner->next; }
      }  current = current->next;
   }}
// Display
void display() {
   Node* temp = head;
   while (temp != NULL) { cout << temp->data << " -> "; temp = temp->next; }
   cout << "NULL" << endl;}
// Main function
int main() {
   insertEnd("A"); insertEnd("B"); insertEnd("A");
   insertEnd("C"); insertEnd("B"); insertEnd("D"); insertEnd("A");
   cout << "Original: "; display();
   removeDuplicates();
   cout << "After removing duplicates: "; display();
```

```
    return 0;
}
```

```
Original: A -> B -> A -> C -> B -> D -> A -> NULL
After removing duplicates: A -> B -> C -> D -> NULL

--------------------------------
Process exited after 0.5389 seconds with return value 0
Press any key to continue . . .
```

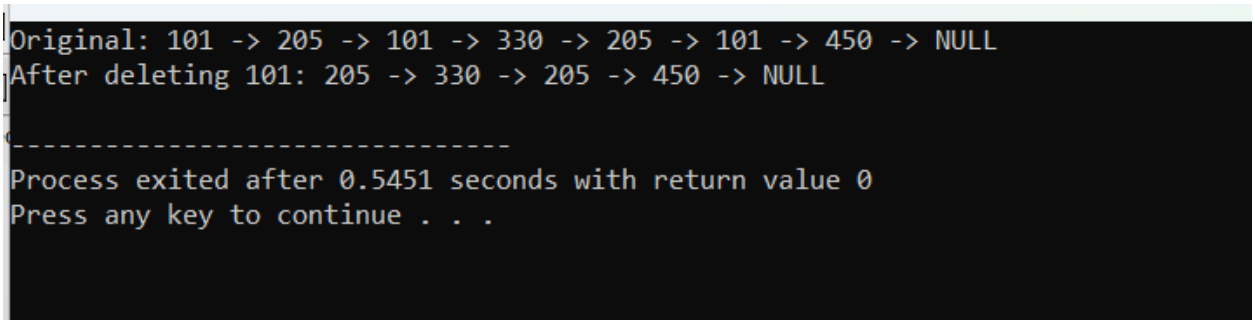
Task 9: Delete All Instances of a Given Book ID

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
    int data;      // Book ID
    Node* next;
};
Node* head = NULL;
// Insert at end
void insertEnd(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
```

```
    temp->next = newNode;
}
// Delete all instances of a given value
void deleteAllInstances(int value) {
    // Step 1: Handle head nodes
    while (head != NULL && head->data == value) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    if (head == NULL) return;
    // Step 2: Handle rest of the list
    Node* current = head;
    while (current->next != NULL) {
        if (current->next->data == value) {
            Node* duplicate = current->next;
            current->next = duplicate->next;
            delete duplicate;
        } else {
            current = current->next;
        }
    }
}

// Display
void display() {
```

```
    Node* temp = head;
    while (temp != NULL) { cout << temp->data << " -> "; temp = temp->next; }
    cout << "NULL" << endl;
}
// Main function
int main() {
    insertEnd(101); insertEnd(205); insertEnd(101); insertEnd(330);
    insertEnd(205); insertEnd(101); insertEnd(450);
    cout << "Original: "; display();
    deleteAllInstances(101);
    cout << "After deleting 101: "; display();
    return 0;
}
```



Task 10: Swap Every Two Consecutive Nodes

```
#include <iostream>
using namespace std;
// Node structure
struct Node {
```

```
    int data;
    Node* next;
};
Node* head = NULL;
// Insert at end
void insertEnd(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
}
// Swap every two consecutive nodes
void swapPairs() {
    if (head == NULL || head->next == NULL) return;  // Nothing to swap
    Node* current = head;
    head = head->next;      // New head is second node
    Node* prev = NULL;
    while (current != NULL && current->next != NULL) {
        Node* first = current;         // First node of pair
        Node* second = current->next;   // Second node of pair
        first->next = second->next;     // First points to node after second
        second->next = first;          // Second points to first
        if (prev != NULL) prev->next = second;  // Connect previous pair
```

```
        prev = first;                // Move prev forward
        current = first->next;       // Move current to next pair
    }}
// Display
void display() {
    Node* temp = head;
    while (temp != NULL) { cout << temp->data << " -> "; temp = temp->next; }
    cout << "NULL" << endl;
}
// Main function
int main() {
    insertEnd(1); insertEnd(2); insertEnd(3);
    insertEnd(4); insertEnd(5); insertEnd(6);
    cout << "Original: "; display();
    swapPairs();
    cout << "After swapping pairs: "; display();
    return 0;
}
```

```
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL
After swapping pairs: 2 -> 1 -> 4 -> 3 -> 6 -> 5 -> NULL

--------------------------------
Process exited after 0.3558 seconds with return value 0
Press any key to continue . . .
```

Doubly Linked List

Task 1: Swap Desks (First with Last, Second with Second-Last, etc.)

```
#include <iostream>
#include <string>
using namespace std;
// Node structure for Doubly Linked List
struct Node {
   string data;    // Stores the desk name (e.g., "Alice")
   Node* next;     // Points to the next node
   Node* prev;     // Points to the previous node
};
Node* head = NULL;  // Global head pointer, initially NULL
Node* tail = NULL;  // Global tail pointer, initially NULL
// Insert a new node at the end of the list
void insertEnd(string value) {
   Node* newNode = new Node;   // Create a new node in memory
   newNode->data = value;      // Store the value in the new node
   newNode->next = NULL;       // New node's next is NULL
   newNode->prev = NULL;       // New node's prev is NULL
   if (head == NULL) {         // If list is empty
      head = newNode;          // Make new node the head
      tail = newNode;          // Make new node the tail
      return;                  // Exit the function
   }
   tail->next = newNode;       // Old tail points to new node
   newNode->prev = tail;       // New node points back to old tail
```

```
    tail = newNode;            // Update tail to new node
}
// Display the list from head to tail
void display() {
    Node* temp = head;         // Start from head
    while (temp != NULL) {     // Traverse the list
        cout << temp->data << " <-> ";  // Print current node's data
        temp = temp->next;     // Move to next node    }
    cout << "NULL" << endl;    // End of list
}
// Swap desks from both ends toward the center
void swapEndsTowardCenter() {
    Node* left = head;         // Left pointer starts at head
    Node* right = tail;        // Right pointer starts at tail
    // Loop until left and right meet or cross
    while (left != right && left->prev != right) {
        // Swap the data of left and right nodes
       string temp = left->data;   // Store left data temporarily
        left->data = right->data;   // Copy right data to left
        right->data = temp;        // Copy stored data to right
        left = left->next;     // Move left pointer forward
        right = right->prev;   // Move right pointer backward
    }
}
// Main function
int main() {
```

```
    insertEnd("Alice");     // Insert Alice at end
    insertEnd("Bob");       // Insert Bob at end
    insertEnd("Charlie");   // Insert Charlie at end
    insertEnd("Dana");      // Insert Dana at end
    insertEnd("Eva");       // Insert Eva at end
    insertEnd("Frank");     // Insert Frank at end
    cout << "Original: "; display();    // Display original list
    swapEndsTowardCenter();            // Swap desks
    cout << "After swapping: "; display(); // Display swapped list
    return 0;
}
```

```
Original: Alice <-> Bob <-> Charlie <-> Dana <-> Eva <-> Frank <-> NULL
After swapping: Frank <-> Eva <-> Dana <-> Charlie <-> Bob <-> Alice <-> NULL

--------------------------------
Process exited after 0.4425 seconds with return value 0
Press any key to continue . . .
```

Task 2: Music Player Playlist (Doubly Linked List)

```
#include <iostream>
#include <string>
using namespace std;
// Node structure for Doubly Linked List
struct Node {
    string song;    // Stores the song name
    Node* next;     // Points to the next song
```

```
    Node* prev;     // Points to the previous song
};
Node* head = NULL;  // Global head pointer
Node* tail = NULL;  // Global tail pointer

// Add a new song at the end of the playlist
void addSong(string name) {
    Node* newNode = new Node;   // Create a new node
    newNode->song = name;       // Store the song name
    newNode->next = NULL;       // Next is NULL
    newNode->prev = NULL;       // Prev is NULL
    if (head == NULL) {         // If playlist is empty
        head = newNode;         // New node becomes head
        tail = newNode;         // New node becomes tail
        return;
    }
    tail->next = newNode;       // Old tail points to new node
    newNode->prev = tail;       // New node points back to old tail
    tail = newNode;             // Update tail
}
// Play songs forward (head to tail)
void playForward() {
    Node* temp = head;          // Start from head
    cout << "Playing forward: ";
    while (temp != NULL) {      // Traverse forward
        cout << temp->song << " -> ";   // Print current song
```

```
        temp = temp->next;      // Move to next song
    }
    cout << "END" << endl;
}
// Play songs backward (tail to head)
void playBackward() {
    Node* temp = tail;          // Start from tail
    cout << "Playing backward: ";
    while (temp != NULL) {      // Traverse backward
        cout << temp->song << " -> ";   // Print current song
        temp = temp->prev;      // Move to previous song
    } cout << "START" << endl;
}
// Main function
int main() {
    addSong("Song A");      // Add Song A
    addSong("Song B");      // Add Song B
    addSong("Song C");      // Add Song C
  addSong("Song D");      // Add Song D
    playForward();          // Play forward
    playBackward();         // Play backward
    return 0; }
```

```
Playing forward: Song A -> Song B -> Song C -> Song D -> END
Playing backward: Song D -> Song C -> Song B -> Song A -> START

--------------------------------
Process exited after 0.7385 seconds with return value 0
Press any key to continue . . .
```

Task 3: Inventory Management System (Nested Linked Lists)

```
#include <iostream>
#include <string>
using namespace std;
// Item node (product in a section)
struct Item {
    string name;    // Item name (e.g., "Teddy Bear")
    Item* next;     // Next item in the same section
};
// Section node (section in a store)
struct Section {
    string name;    // Section name (e.g., "Toys")
    Section* next;  // Next section in the same store
    Item* items;    // Head of items list in this section
};
// Store node (a store in the chain)
struct Store {
    string name;    // Store name (e.g., "Store A")
    Store* next;    // Next store in the chain
    Section* sections;  // Head of sections list in this store
};
Store* storeHead = NULL;    // Head of the store list

// Add a new store to the chain
void addStore(string name) {
    Store* newStore = new Store;    // Create new store
```

```
    newStore->name = name;         // Store the name
    newStore->next = NULL;         // Next is NULL
    newStore->sections = NULL;     // No sections yet
    if (storeHead == NULL) {       // If no stores exist
        storeHead = newStore;      // New store becomes head
        return;
    }
    Store* temp = storeHead;       // Start from head
    while (temp->next != NULL) temp = temp->next;   // Go to last store
    temp->next = newStore;         // Link last store to new store
}
// Add a new section to a specific store
void addSection(string storeName, string sectionName) {
    Store* s = storeHead;          // Start from store head
    while (s != NULL && s->name != storeName) s = s->next;  // Find store
    if (s == NULL) { cout << "Store not found" << endl; return; }
    Section* newSec = new Section;  // Create new section
    newSec->name = sectionName;    // Store name
    newSec->next = NULL;           // Next is NULL
    newSec->items = NULL;          // No items yet

    if (s->sections == NULL) {     // If no sections in store
        s->sections = newSec;      // New section becomes head
    } else {
        Section* temp = s->sections;    // Start from first section
        while (temp->next != NULL) temp = temp->next;   // Go to last
```

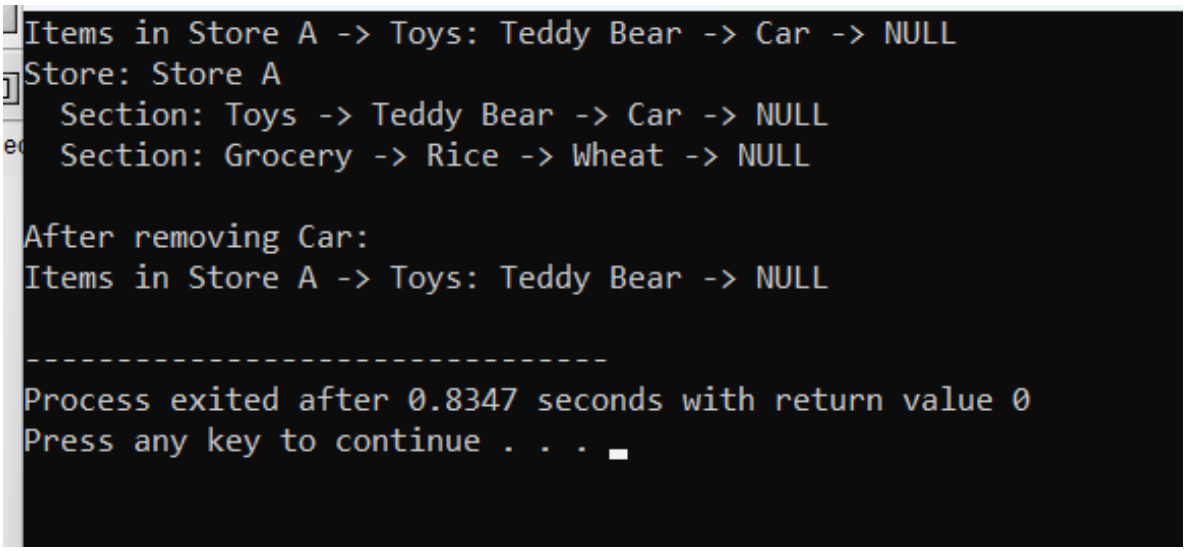
```
        temp->next = newSec;        // Link last section to new one
    }
}
// Add an item to a specific section of a specific store
void addItem(string storeName, string sectionName, string itemName) {
    Store* s = storeHead;           // Find store
    while (s != NULL && s->name != storeName) s = s->next;
    if (s == NULL) { cout << "Store not found" << endl; return; }
    Section* sec = s->sections;     // Find section
    while (sec != NULL && sec->name != sectionName) sec = sec->next;
    if (sec == NULL) { cout << "Section not found" << endl; return; }
    Item* newItem = new Item;       // Create new item
    newItem->name = itemName;       // Store item name
    newItem->next = NULL;           // Next is NULL
    if (sec->items == NULL) {       // If no items in section
        sec->items = newItem;       // New item becomes head
    } else {
        Item* temp = sec->items;    // Start from first item
        while (temp->next != NULL) temp = temp->next;   // Go to last
        temp->next = newItem;       // Link last item to new one
    }
}
// Remove an item from a specific section of a specific store
void removeItem(string storeName, string sectionName, string itemName) {
    Store* s = storeHead;           // Find store
    while (s != NULL && s->name != storeName) s = s->next;
```

```
    if (s == NULL) return;
    Section* sec = s->sections;     // Find section
    while (sec != NULL && sec->name != sectionName) sec = sec->next;
    if (sec == NULL) return;
    Item* current = sec->items;     // Start from first item
    Item* prev = NULL;              // Previous item pointer
    while (current != NULL && current->name != itemName) {
        prev = current;             // Move prev forward
        current = current->next;    // Move current forward
    }  if (current == NULL) { cout << "Item not found" << endl; return; }

    if (prev == NULL) sec->items = current->next;   // Remove head item
    else prev->next = current->next;                // Remove middle item
    delete current;                 // Free memory
}
// Display all items of a specific section of a store
void displaySection(string storeName, string sectionName) {
    Store* s = storeHead;           // Find store
    while (s != NULL && s->name != storeName) s = s->next;
    if (s == NULL) return;
    Section* sec = s->sections;     // Find section
    while (sec != NULL && sec->name != sectionName) sec = sec->next;
    if (sec == NULL) return;
    cout << "Items in " << storeName << " -> " << sectionName << ": ";
    Item* temp = sec->items;        // Start from first item
    while (temp != NULL) {          // Traverse items
```

```
        cout << temp->name << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;}
// Display all items of a given store (all sections)
void displayStore(string storeName) {
    Store* s = storeHead;          // Find store
    while (s != NULL && s->name != storeName) s = s->next;
    if (s == NULL) return;
    cout << "Store: " << storeName << endl;
    Section* sec = s->sections;    // Start from first section
    while (sec != NULL) {          // Traverse sections
        cout << "  Section: " << sec->name << " -> ";
        Item* temp = sec->items;   // Start from first item
        while (temp != NULL) {     // Traverse items
            cout << temp->name << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
        sec = sec->next;      }   }  // Move to next section
// Main function
int main() {
    addStore("Store A");                          // Add store
    addSection("Store A", "Toys");                    // Add section
    addSection("Store A", "Grocery");                  // Add section
    addItem("Store A", "Toys", "Teddy Bear");             // Add item
```

```
    addItem("Store A", "Toys", "Car");                  // Add item
    addItem("Store A", "Grocery", "Rice");              // Add item
    addItem("Store A", "Grocery", "Wheat");             // Add item
    displaySection("Store A", "Toys");                  // Display
    displayStore("Store A");                            // Display
    removeItem("Store A", "Toys", "Car");               // Remove item
    cout << "\nAfter removing Car:" << endl;
    displaySection("Store A", "Toys");                  // Display again
    return 0;
}
```



Task 4: Theater Seating Arrangement

```
#include <iostream>

using namespace std;
```

```
// Node structure for Doubly Linked List
struct Node {
   int data;      // Seat number
   Node* next;    // Points to next seat
   Node* prev;    // Points to previous seat
};

Node* head = NULL;  // Global head pointer
Node* tail = NULL;  // Global tail pointer
// Insert a new seat at the end
void insertEnd(int value) {
   Node* newNode = new Node;   // Create a new node
   newNode->data = value;      // Store the seat number
   newNode->next = NULL;       // Next is NULL
   newNode->prev = NULL;       // Prev is NULL
   if (head == NULL) {         // If list is empty
      head = newNode;          // New node becomes head
      tail = newNode;          // New node becomes tail
      return;
   }
   tail->next = newNode;       // Old tail points to new node
   newNode->prev = tail;       // New node points back to old tail
   tail = newNode;             // Update tail
}
```

```
// Display the list
void display() {
    Node* temp = head;          // Start from head
    while (temp != NULL) {      // Traverse forward
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
// Swap the data of two nodes
void swapData(Node* a, Node* b) {
    int temp = a->data;     // Store a's data
    a->data = b->data;      // Copy b's data to a
    b->data = temp;         // Copy stored data to b
}
// Theater seating arrangement
void theaterArrange() {
    Node* left = head->next;    // Start from 2nd seat (skip first)
    Node* right = tail->prev;   // Start from 2nd-last seat (skip last)
    bool doSwap = true;         // Flag: first pair needs to be swapped
    // Loop until left and right meet or cross
    while (left != right && left->prev != right) {
        if (doSwap) {           // If this pair needs swapping
            swapData(left, right);  // Swap the values
        }
        left = left->next;      // Move left forward
```

```
        right = right->prev;    // Move right backward
        doSwap = !doSwap;       // Alternate swap flag
    }
}
// Main function
int main() {
    // Create seats 1 to 9
    for (int i = 1; i <= 9; i++) insertEnd(i);
    cout << "Original: "; display();    // Display original
    theaterArrange();                   // Arrange seats
    cout << "After arrangement: "; display(); // Display result
    return 0;
}
```

```
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> NULL
After arrangement: 1 -> 8 -> 3 -> 6 -> 5 -> 4 -> 7 -> 2 -> 9 -> NULL

--------------------------------
Process exited after 0.3515 seconds with return value 0
Press any key to continue . . .
```

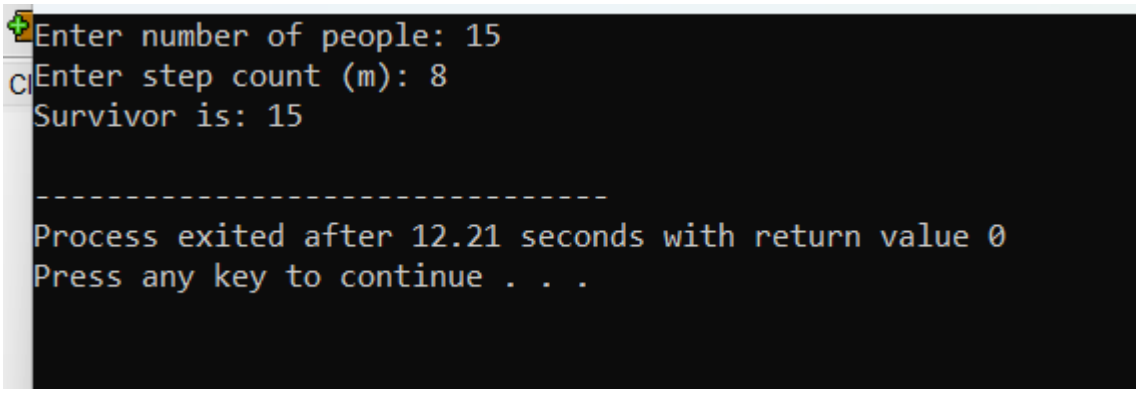
Circular Linked List

Task 1: Josephus Problem

```
#include <iostream>
using namespace std;
// Node structure for Circular Linked List
struct Node {
   int data;      // Stores the person's number
   Node* next;    // Points to the next person in the circle
};
// Function to create a circular linked list with n people
Node* createCircle(int n) {
   Node* head = NULL;     // Head of the circle
   Node* tail = NULL;     // Tail of the circle
   // Loop to create n nodes (1 to n)
   for (int i = 1; i <= n; i++) {
      Node* newNode = new Node;   // Create a new node
      newNode->data = i;          // Store the person number
      newNode->next = NULL;       // Next is NULL initially
      if (head == NULL) {         // If this is the first node
         head = newNode;          // New node becomes head
         tail = newNode;          // New node becomes tail
         newNode->next = head;    // Point to itself (circle complete)
      } else {
         tail->next = newNode;    // Old tail points to new node
         newNode->next = head;    // New node points back to head
         tail = newNode;          // Update tail to new node
```

```
    }
    }
    return head;    // Return the head of the circle
}
// Function to solve Josephus problem
int josephus(int n, int m) {
    // Step 1: Create circular linked list
    Node* head = createCircle(n);
    // Step 2: Initialize pointers
    Node* current = head;       // Current person being counted
    Node* prev = head;          // Person just before current
    // Step 3: Find the node just before head (last node)
    while (prev->next != head) {
        prev = prev->next;      // Move prev to last node
    }
    // Step 4: Eliminate until only one person remains
    while (current->next != current) {
        // Skip m-1 persons
        for (int count = 1; count < m; count++) {
            prev = current;         // Move prev forward
            current = current->next; // Move current forward
        }
        // Eliminate the m-th person
        prev->next = current->next; // Skip the current node
        delete current;             // Free memory of eliminated person
        current = prev->next;       // Move current to next person
```

```
    }
    // Step 5: Return survivor's number
    int survivor = current->data;   // Store survivor's data
    delete current;               // Free last node
    return survivor;              // Return survivor number
}
// Main function
int main() {
    int n, m;
    cout << "Enter number of people: ";
    cin >> n;                    // Read number of people
    cout << "Enter step count (m): ";
    cin >> m;                     // Read step count
    int survivor = josephus(n, m);  // Call Josephus function
    cout << "Survivor is: " << survivor << endl;    // Print result
    return 0;
}
```



Task 2: Task Scheduling System (Round-Robin)

```
#include <iostream>
#include <string>
using namespace std;
// Node structure for Circular Linked List
struct Task {
    string name;       // Stores the task name
    int priority;      // Stores the task priority
    string status;     // pending, in-progress, completed
    Task* next;        // Points to the next task in the circle
};
Task* head = NULL;     // Global head pointer
// Add a new task at the end of the circular list
void addTask(string name, int priority) {
    Task* newTask = new Task;   // Create a new task
    newTask->name = name;       // Store the task name
    newTask->priority = priority;   // Store the priority
    newTask->status = "pending";    // Default status is pending
    newTask->next = NULL;       // Next is NULL
    if (head == NULL) {         // If list is empty
        head = newTask;         // New task becomes head
        newTask->next = head;   // Point to itself (circle complete)
        return;
    }
    // Traverse to the last task
    Task* temp = head;
```

```
    while (temp->next != head) {
        temp = temp->next;      // Move to next task
    }
    temp->next = newTask;       // Old last task points to new task
    newTask->next = head;       // New task points back to head
}
// Remove a task by its name
void removeTask(string name) {
    if (head == NULL) {         // If list is empty
        cout << "List is empty" << endl;
        return;
    }
    Task* current = head;       // Start from head
    Task* prev = NULL;          // Previous task pointer
    // Find the node just before head (last node)
    prev = head;
    while (prev->next != head) {
        prev = prev->next;
    }
    // Case 1: Only one node exists
    if (head->next == head && head->name == name) {
        delete head;            // Delete the only node
        head = NULL;            // List becomes empty
        return;
    }
    // Case 2: Head node needs to be removed
```

```
    if (head->name == name) {
        prev->next = head->next;    // Last node points to second node
        Task* temp = head;          // Store old head
        head = head->next;          // Move head to second node
        delete temp;                // Delete old head
        return;
    }
    // Case 3: Middle or last node
    current = head;
    while (current->next != head && current->next->name != name) {
        current = current->next;    // Move forward
    }
    if (current->next->name == name) {
        Task* temp = current->next;         // Store task to delete
        current->next = temp->next;         // Skip the task
        delete temp;                        // Delete the task
    } else {
        cout << "Task not found" << endl;
    }
}
// Get next pending task in round-robin fashion
Task* getNextTask(Task* current) {
    if (current == NULL) return NULL;       // If no current task
    Task* temp = current->next;             // Start from next task
    // Skip completed tasks
    while (temp != current && temp->status == "completed") {
```

```
        temp = temp->next;                // Move forward
    }
    if (temp->status == "completed") {     // All tasks completed
        cout << "No pending tasks found" << endl;
        return NULL;}
    return temp;          }               // Return next pending task
// Display all tasks in the circular list
void displayTasks() {
    if (head == NULL) {                   // If list is empty
        cout << "List is empty" << endl;
        return;}
    Task* temp = head;                    // Start from head
    cout << "Tasks:" << endl;
    // Use do-while because we need to print first node too
    do {
        cout << "  Name: " << temp->name
             << ", Priority: " << temp->priority
             << ", Status: " << temp->status << endl;
        temp = temp->next;                // Move to next task
    } while (temp != head);               // Stop when we return to head
}
// Update task status by name
void updateStatus(string name, string newStatus) {
    if (head == NULL) return;             // If list is empty
    Task* temp = head;                    // Start from head
    // Traverse the circular list
```

```
    do {
        if (temp->name == name) {          // If task found
            temp->status = newStatus;      // Update status
            cout << "Task '" << name << "' status updated to: " << newStatus << endl;
            return;                        // Exit function
        }
        temp = temp->next;                 // Move forward
    } while (temp != head);                // Stop when back to head
    cout << "Task not found" << endl;      // Task not found
}
// Main function
int main() {
    addTask("Design", 1);          // Add task
    addTask("Development", 2);     // Add task
    addTask("Testing", 3);         // Add task
    addTask("Deployment", 4);      // Add task
    cout << "=== All Tasks ===" << endl;
    displayTasks();                // Display all tasks
    updateStatus("Design", "completed");    // Update status
    cout << "\n=== After updating Design ===" << endl;
    displayTasks();                // Display again
    removeTask("Testing");         // Remove task
    cout << "\n=== After removing Testing ===" << endl;
    displayTasks();                // Display again
    return 0;
}
```

```
=== All Tasks ===
Tasks:
  Name: Design, Priority: 1, Status: pending
  Name: Development, Priority: 2, Status: pending
  Name: Testing, Priority: 3, Status: pending
  Name: Deployment, Priority: 4, Status: pending
Task 'Design' status updated to: completed

=== After updating Design ===
Tasks:
  Name: Design, Priority: 1, Status: completed
  Name: Development, Priority: 2, Status: pending
  Name: Testing, Priority: 3, Status: pending
  Name: Deployment, Priority: 4, Status: pending

=== After removing Testing ===
Tasks:
  Name: Design, Priority: 1, Status: completed
  Name: Development, Priority: 2, Status: pending
  Name: Deployment, Priority: 4, Status: pending

--------------------------------
Process exited after 0.6943 seconds with return value 0
Press any key to continue . . .
```